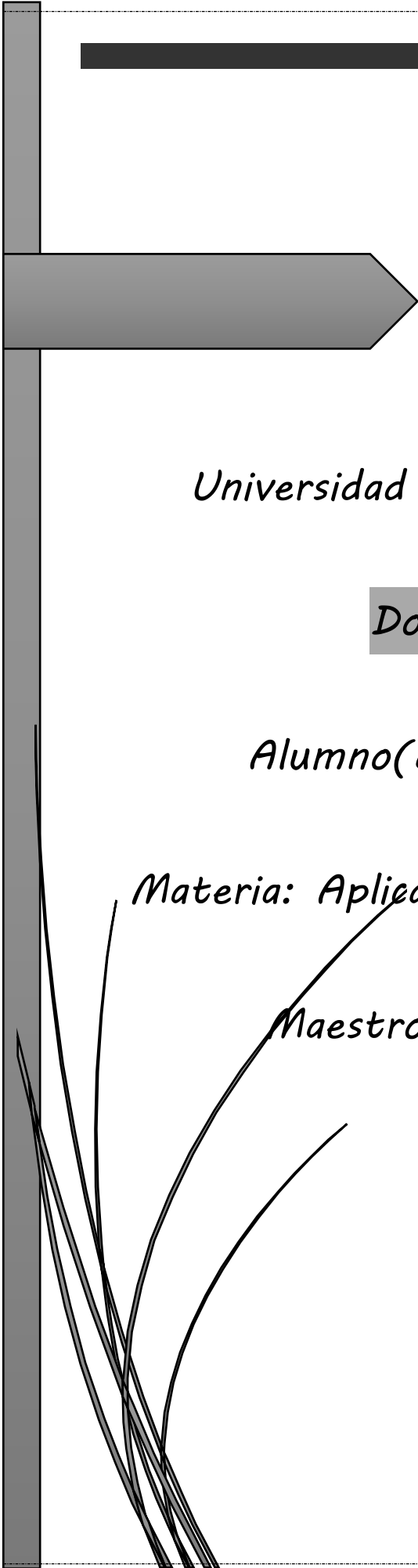




Universidad Tecnológica De Nezahualcóyotl

Documentación Render

Alumno(a): Solis Garcia Fany Sarai

Materia: Aplicaciones Web Orientada a Objetos

Maestro: Del Prado López Jovan

¿Qué hace esta página?

Es un visor interactivo que muestra una sudadera desde diferentes ángulos. El usuario arrastra la imagen con el mouse o con el dedo, y la sudadera gira o cambia de perspectiva según hacia dónde se mueva. No hay video ni animación real: lo que hace el código es cambiar rápidamente entre **20 fotografías estáticas** que fueron tomadas desde distintos ángulos, creando la ilusión de que la prenda rota. También tiene un botón para ver el render en pantalla completa.

2 Estructura HTML

La página es un solo archivo HTML. No tiene formularios ni múltiples secciones, solo el título, el contenedor del visor y el texto de ayuda abajo.

Elemento	Id / Clase	Para qué sirve
div	.header	Contiene el título 'Render Sudadera' arriba de todo
div	#contenedor-principal	Caja de 550x550px que engloba el visor y el botón de fullscreen
div	.controls-top-right	Posiciona el botón de pantalla completa en la esquina superior derecha del contenedor
button	#btn-fullscreen	Botón con el emoji de patita que activa la pantalla completa
div	#visor	El área donde se muestra la imagen; es quien detecta el arrastre del usuario
img	#producto	La imagen que cambia según el movimiento. Empieza con gato5.png
p	.hint-text	Texto pequeño abajo que dice 'Arrastra para rotar la imagen'

⚠ La imagen inicial es gato5.png porque las variables empiezan en col=1, row=2, y la fórmula da $n = 4 + 1 = 5$. No es un error, es el ángulo frontal de la sudadera.

3 Estilos CSS

body — Centrado y fondo blanco

```
body { display: flex; flex-direction: column; justify-content: center; align-items: center; height: 100vh; overflow: hidden; }
```

El body usa Flexbox en columna para centrar todo (título + visor + texto) tanto horizontal como verticalmente en la ventana. overflow:hidden evita que aparezcan barras de desplazamiento si algo se sale unos píxeles.

#contenedor-principal — Caja del visor

```
#contenedor-principal { position: relative; width: 550px; height: 550px; }
```

Es un cuadrado fijo de 550x550 px. La propiedad relative es necesaria para que el botón de pantalla completa, que tiene position:absolute, se posicione dentro de esta caja y no en toda la pantalla.

#btn-fullscreen — Botón circular

```
#btn-fullscreen { border-radius: 50%; width: 50px; height: 50px; position: absolute en .controls-top-right }
```

El botón es un círculo blanco con sombra suave. Tiene una transición de scale al pasar el cursor para dar feedback visual. El emoji de patita funciona como ícono sin necesidad de ninguna librería externa.

:fullscreen — Ajuste en pantalla completa

```
:fullscreen #contenedor-principal { width: 100vw; height: 100vh; }
```

Cuando el usuario activa la pantalla completa, el contenedor se expande para ocupar toda la ventana. La imagen se agranda automáticamente porque tiene max-width:100%.

#visor — Cursor de movimiento

```
#visor { cursor: move; touch-action: none; }
```

cursor:move cambia el cursor a la flecha de cuatro direcciones para indicar que se puede arrastrar. touch-action:none desactiva el comportamiento táctil por defecto del navegador (como hacer scroll) para que el código propio pueda manejar el toque sin interferencias.

4 Las 20 Imágenes — Cómo están organizadas

Las 20 imágenes no son aleatorias. Están distribuidas en una cuadrícula virtual de **3 filas y hasta 12 columnas**. La fila del medio tiene 12 ángulos (una vuelta completa horizontal), y las filas de arriba y abajo tienen 4 ángulos cada una para la perspectiva vertical.

Fila (row)	Columnas disponibles	Números de imagen	Vista
1	1 — 4	gato1.png — gato4.png	Vista superior / ángulo alto
2	1 — 12	gato5.png — gato16.png	Vista frontal — rotación completa (12 ángulos)
3	1 — 4	gato17.png — gato20.png	Vista inferior / ángulo bajo

Cómo se calcula el número de imagen

La función `update()` usa una fórmula simple que depende de en qué fila y columna está el usuario:

```
if (row === 1) n = col;           // gato1 a gato4
if (row === 2) n = 4 + col;       // gato5 a gato16
if (row === 3) n = 16 + col;      // gato17 a gato20
```

⚠ Las imágenes deben estar en una carpeta llamada `fotos/` y nombrarse exactamente `gato1.png`, `gato2.png`... hasta `gato20.png`. Si falta alguna o el nombre es diferente, esa posición queda en blanco.

5 Lógica JavaScript

5.1 Variables de estado

```
let col = 1;    // columna actual (posición horizontal)
let row = 2;    // fila actual (posición vertical)
let dragging = false; // si el usuario está arrastrando
let startX, startY; // punto donde empezó el arrastre
```

Estas cuatro variables guardan todo el estado del visor. `col` y `row` dicen en qué imagen estamos, `dragging` dice si el usuario está moviendo el mouse o el dedo en este momento, y `startX/startY` guardan el punto de inicio del arrastre para calcular cuánto se movió.

5.2 Función `update()`

```
function update() {
  let n = 1;
  if (row === 1) {
    if (col > 4) col = 1; n = col;
  } else if (row === 2) {
    n = 4 + col;
  } else {
    if (col > 4) col = 1; n = 16 + col;
  }
  img.src = `fotos/gato${n}.png`;
}
```

Calcula qué imagen mostrar con base en `row` y `col`, y le asigna la ruta al atributo `src` de la imagen. El navegador carga esa imagen inmediatamente. También ajusta `col` si se pasa del máximo de columnas disponibles en esa fila.

5.3 Función `mover(currentX, currentY)`

```
const mover = (currentX, currentY) => {
  let diffX = currentX - startX;
  let diffY = currentY - startY;
  let sensX = 25; // pixeles para cambiar columna
  let sensY = 55; // pixeles para cambiar fila
  if (Math.abs(diffX) > sensX) {
    let maxCol = (row === 2) ? 12 : 4;
    if (diffX > 0) col--; else col++;
    startX = currentX; // reinicia el punto de inicio
    update();
  }
  if (Math.abs(diffY) > sensY) {
    if (diffY > 0 && row > 1) row--;
    else if (diffY < 0 && row < 3) row++;
    startY = currentY;
    update();
  }
};
```

Esta es la función central. Compara la posición actual del mouse con la posición de inicio. Si la diferencia horizontal supera 25 px, cambia la columna (rota la sudadera). Si la diferencia vertical supera 55 px, cambia la fila (cambia el ángulo de altura). Cada vez que se registra un cambio, el punto de inicio se actualiza para que el siguiente movimiento se mida desde ahí.

5.4 Eventos de mouse y touch

```
// Mouse
visor.onmousedown = (e) => {
  dragging = true;
  startX = e.clientX;
  startY = e.clientY;
};
window.onmousemove = (e) => mover(e.clientX, e.clientY);
window.onmouseup = () => {
  dragging = false;
};
// Táctil
visor.ontouchstart = (e) => {
  dragging = true;
  startX = e.touches[0].clientX;
  ...
};
visor.ontouchmove = (e) => mover(e.touches[0].clientX, e.touches[0].clientY);
window.ontouchend = () => dragging = false;
```

Los eventos de mouse funcionan en computadora y los de touch en celular o tablet. `onmousedown` y `ontouchstart` activan el arrastre y guardan el punto inicial. `onmousemove` y `ontouchmove` llaman a `mover()` mientras el usuario arrastra. `onmouseup` y `ontouchend` desactivan el arrastre. El



mousemove se escucha en window (no en el visor) para que si el cursor sale del área del visor el arrastre no se corte.

5.5 Precarga de imágenes

```
| for (let i = 1; i <= 20; i++) {   const p = new Image();   p.src = `fotos/gato${i}.png`; }
```

Al cargar la página, este loop crea 20 objetos Image y les asigna la ruta de cada foto. El navegador las descarga en segundo plano sin mostrarlas. Cuando el usuario arrastra y update() cambia el src, la imagen ya está en memoria y aparece instantáneamente sin parpadear ni tardar.

6 Pantalla Completa

El botón con la patita activa la API nativa de pantalla completa del navegador. No requiere librerías externas.

```
btnFull.addEventListener('click', () => {      if (!document.fullscreenElement) {  
  container.requestFullscreen(); // entra en fullscreen  } else {      document.exitFullscreen();  
  // sale de fullscreen  } });
```

requestFullscreen() le pide al navegador que expanda el elemento **#contenedor-principal** a toda la pantalla. Si ya está en pantalla completa, exitFullscreen() lo regresa al tamaño normal. El CSS tiene una regla :fullscreen que hace que el contenedor se expanda a 100vw x 100vh automáticamente.

⚠ La API Fullscreen funciona en todos los navegadores modernos. En Safari de iOS tiene ciertas limitaciones y puede que no funcione en todos los dispositivos Apple.

7 Flujo de Interacción

#	Qué pasa
1	La página carga y el loop precarga las 20 imágenes en memoria
2	Se muestra gato5.png (row=2, col=1) como imagen inicial
3	El usuario presiona el mouse o toca la pantalla sobre el visor
4	Se activa dragging=true y se guardan las coordenadas de inicio
5	El usuario mueve el mouse/dedo hacia la derecha
6	mover() detecta que diffX supera 25 px y decrementa col
7	update() calcula el nuevo n y cambia img.src a la siguiente imagen
8	La imagen cambia instantáneamente porque ya estaba precargada
9	El usuario mueve hacia abajo más de 55 px
10	mover() incrementa row (cambia de fila) y llama a update()
11	La sudadera ahora se ve desde un ángulo más bajo
12	El usuario suelta el mouse/dedo: dragging vuelve a false